

Freight Rate Prediction

Machine Learning Engineer assessment · model and validation report

October 2, 2026

Summary

- **Model.** A two-stage hybrid on $\log(\text{rate})$: a robust log-linear trend model (distance curve, equipment and weight premiums, `market_index`, calendar drift), LightGBM on its residuals (quarter-end ramp, geography, `quote_signal` deviation) and a shrunk per-lane correction.
- **Validation.** Expanding-window, out-of-time backtests with two-month test windows that mirror the November–December target. On the latest fold (train Jan–Aug, test Sep–Oct) the model scores 1.35% MAPE on clean labels (3.87% including corrupted labels), against 3.45% for a lane-median baseline and 2.42% for LightGBM with a time feature.
- **Data quality.** 677 corrupted labels (1.4%) removed from training, sign-flipped and missing weights repaired, missing `market_index` filled from the same day, a drifting `quote_signal` neutralised, and 8 cities that appear only in validation (12.1% of loads) handled through coordinates.
- **December.** For the fixed Lexington → Fort Wayne load the predicted rate rises from \$826 on Dec 1 to \$869 on Dec 31: a weekly market cycle on top of a year-end quarter-close ramp and a slow upward drift.

1. What the data shows

48,000 labelled loads from 2025-01-01 to 2025-10-31 (`train_test.csv`) and 12,000 unlabelled loads from 2025-11-01 to 2025-12-31 (`validation.csv`). Every driver acts as a percentage on the rate, so all modelling is done on $\log(\text{rate})$.

- **Distance and equipment set the base rate.** Rate per mile falls smoothly from about \$2.87 under 150 miles to \$1.87 beyond 2,500 miles. Reefer and Flatbed carry a near-constant premium of 12.9% and 8.1% over Dry Van; heavier loads add 3.0% per 10,000 lb (Figure 1).
- **`market_index` is a daily, market-wide signal.** It follows a seasonal curve (monthly mean 0.93 in January, 1.30 in May, 0.89 in September) plus a strong weekly cycle (Thursday 1.18 vs Sunday 0.99). Per-load values differ from the day's level by only about 0.025. Rates move 1.45% per 0.1 of index (Figure 2).
- **Prices drift up about 7.9% a year beyond the market.** Same-day `market_index` plus a linear drift explain 97% of the day-to-day price level outside quarter-end windows (Figure 2, lower panel).
- **Quarter-end ramp.** Rates climb over the last three weeks of every quarter, reaching +8.0% for Flatbed, +5.1% for Reefer and +2.7% for Dry Van in the final three days. Ordinary month-ends show no effect (−0.07%), and no holiday effect is visible around New Year, Memorial Day, July 4 or Labor Day (Figure 3).
- **Geography.** City price effects (standard deviation 1.8%) track latitude ($r = -0.84$): southern cities such as Dallas and Oklahoma City are pricier than northern ones such as Syracuse and Buffalo. Coordinates explain this, so the model uses coordinates rather than city identities (Figure 4).

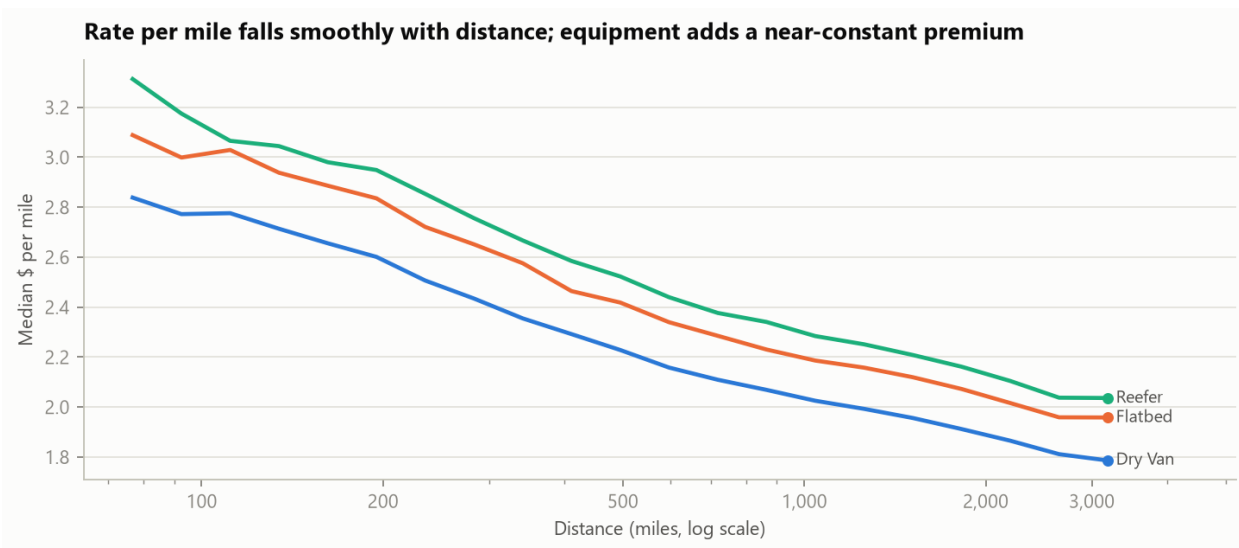


Figure 1. Median rate per mile by distance and equipment (clean labels).

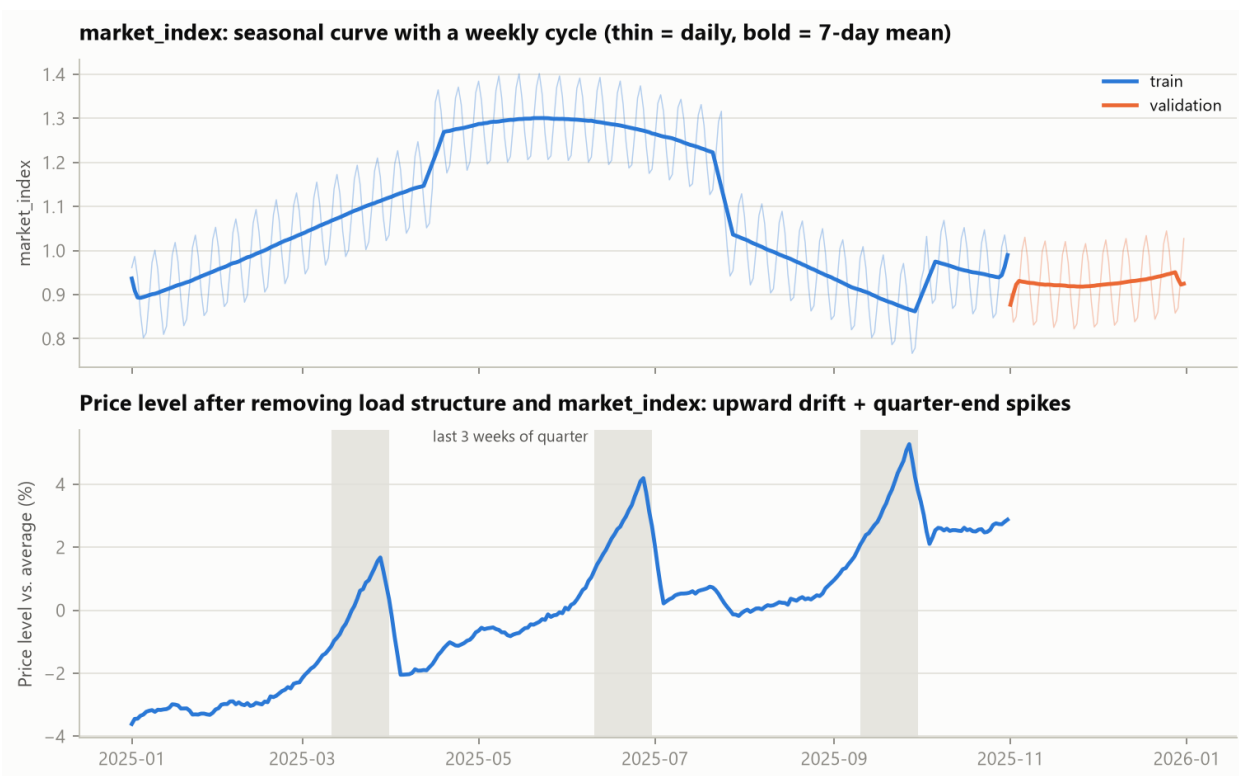


Figure 2. Top: market_index, train and validation. Bottom: price level after removing load structure and market_index; shaded bands are the last three weeks of each quarter.

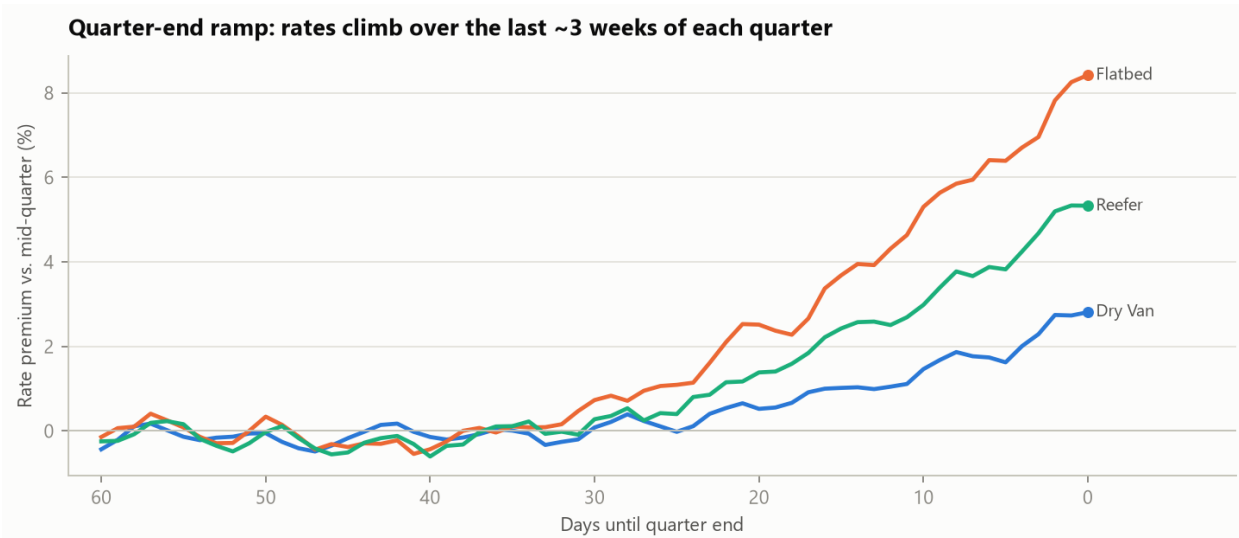


Figure 3. Rate premium relative to mid-quarter, by days until quarter end (after removing structure, market and drift).

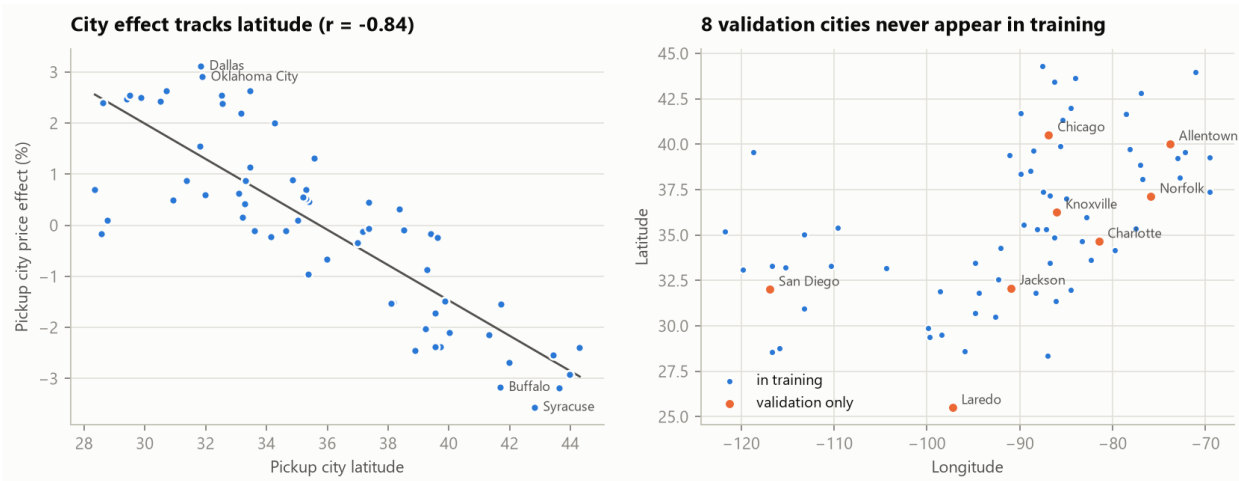


Figure 4. Left: pickup-city price effect against latitude. Right: city coordinates; orange cities appear only in validation.

2. Data-quality issues and how they were handled

Table 1. Data-quality issues. No duplicate rows or inconsistent city coordinates were found.

Issue	Evidence	Treatment
Corrupted labels	677 loads (1.4%) sit at $0.17\text{--}0.46\times$ or $2.2\text{--}5.4\times$ a robust baseline, separated from the clean band ($\pm 10\%$) by empty gaps; spread evenly across months, lanes and equipment (Figure 5).	Flagged with a Huber fit on each training window and removed from training. Kept in evaluation as the "all rows" score.
Negative weights	292 train / 145 validation rows. Their median rate per mile (\$2.16) matches positive-weight loads (\$2.15).	Treated as a sign error: absolute value.
Missing weight	300 train / 165 validation rows.	Equipment median plus a missing-value flag.
Missing market_index	374 train / 249 validation rows.	Median of the same day's loads (within-day spread is only about 0.025).
Capped values	Weight capped at 47,500 lb (1,204 rows); distance floored at 70 miles (48 rows).	Kept as is; they are plausible values at the limits and tree splits handle the flat tails.

quote_signal drift	In training its level jumps with equipment and quarter (Flatbed/Reefer about 2.4 in Mar/Jun/Sep, about 1.75 in other months); in validation it is flat at about 2.05 for all equipment (Figure 6). Using the raw level let the model credit the quarter-end ramp to quote_signal, which erased the ramp in December.	Raw level dropped. Only the deviation from the same day's equipment mean is used (unusually high or low values come with about 1% higher rates).
Unseen cities	8 validation cities (Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego) never appear in training; they touch 12.1% of validation loads.	No city IDs as features; coordinates carry geography. The lane correction is zero for unseen lanes. Checked with a simulated cold-start test (Table 4).
Incomplete December inputs	december_chart_inputs.csv has no coordinates, market_index or quote_signal.	Coordinates from the training city table; market_index and quote_signal set to the median of that day's ~200 validation loads (inputs, not labels).

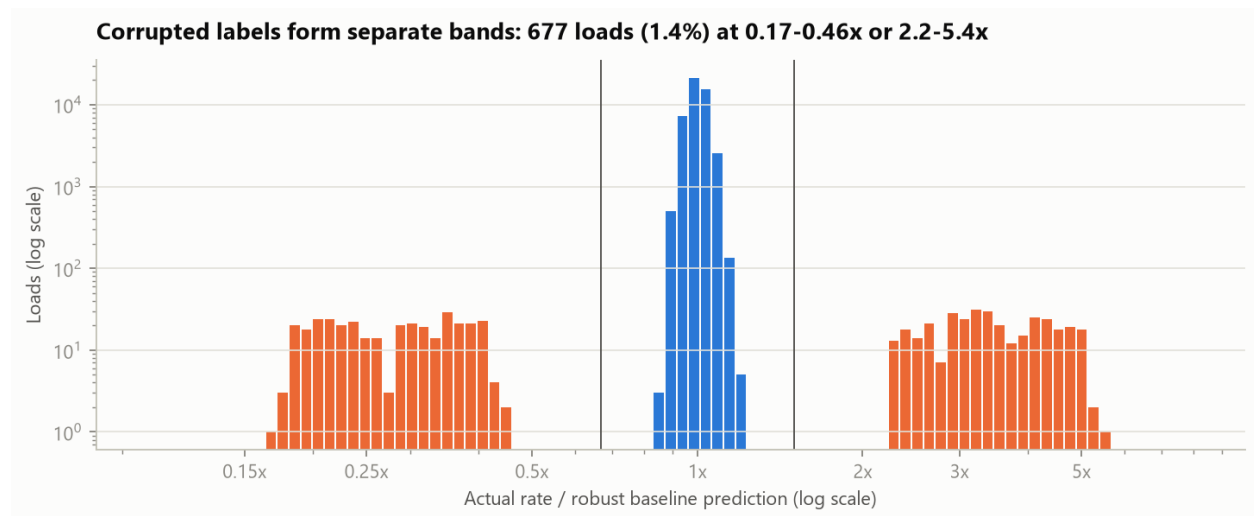


Figure 5. Actual rate divided by a robust baseline prediction. Orange bars (beyond the vertical lines) are treated as corrupted labels.

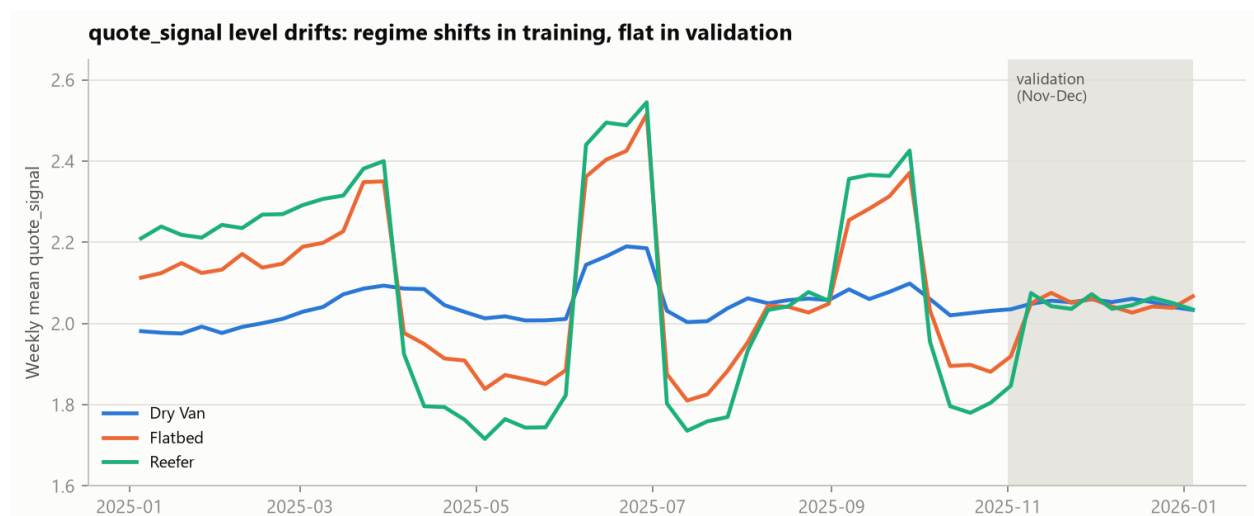


Figure 6. Weekly mean quote_signal by equipment; validation period shaded.

3. Validation approach and data split

The task is a forecast: train on January–October and predict November–December, so the split follows time. A random split would test the model on the same days, market states and quarter-end windows it trained on. A shuffled 5-fold split reports 1.14% MAPE for the final model, flattening the out-of-time figure of 1.35%–1.60%.

Expanding-window backtest

- **Three folds.** Each trains on all data up to a cut-off and tests on the next two months, the same horizon as November–December. Test windows start the day after training ends.
- **Cleaning inside the fold.** Corrupted labels are flagged using the training window only. Test loads are scored on all rows and on clean rows, because no model can predict a 3× label error.
- **Metric.** MAPE is primary: rates range from about \$60 to \$25,000, and percentage error treats short and long hauls alike. MAE, RMSE and median APE are in artifacts/metrics.json. RMSE on all rows is dominated by the corrupted labels (about \$629 vs \$45 on clean rows).
- **Selection rule.** Model and hyperparameter choices use the mean of folds 2 and 3. Fold 1 trains on January–June, when market_index and calendar time rise together, so the trend stage cannot separate them (market coefficient 0.086, drift 0.162/yr). From seven months of data on, both settle (Table 3), and the final model trains on ten.

Table 2. Backtest folds.

Fold	Training window (2025)	Test window (2025)	Training loads (clean)	Test loads
1	Jan 1 – Jun 30	Jul 1 – Aug 31	28,406	9,671
2	Jan 1 – Jul 31	Aug 1 – Sep 30	33,253	9,429
3	Jan 1 – Aug 31	Sep 1 – Oct 31	37,944	9,523

Table 3. Out-of-time MAPE (%), clean rows / all rows. Lower is better.

Model	Fold 1: Jul–Aug	Fold 2: Aug–Sep	Fold 3: Sep–Oct	Mean folds 2–3 (clean)
Lane median \$/mile	3.81 / 6.03	3.55 / 5.79	3.45 / 5.96	3.50
Huber log-linear	4.75 / 7.09	2.82 / 5.07	2.90 / 5.40	2.86
LightGBM	3.70 / 5.88	4.15 / 6.30	3.32 / 5.76	3.74
LightGBM + time feature	2.89 / 5.18	2.47 / 4.71	2.42 / 4.92	2.44
Hybrid: Huber trend + LightGBM	3.90 / 6.23	1.69 / 3.95	1.46 / 3.98	1.57
Hybrid + lane correction (final)	3.88 / 6.21	1.60 / 3.87	1.35 / 3.87	1.47

Table 4. Trend-stage coefficients by training cut-off.

Training data up to	market_index coefficient	Drift per year (log)
Jun 30	0.086	0.162
Jul 31	0.141	0.084
Aug 31	0.160	0.063
Oct 31 (final)	0.144	0.076

Unseen-city test

To check the loads that touch a new city, each of four draws hides 8 random cities from the fold-3 training data and scores the test loads that involve them separately.

Table 5. Simulated cold start, MAPE on clean rows.

Model	Loads with hidden cities	Other loads
Hybrid: Huber trend + LightGBM	1.53%	1.47%
Hybrid + lane correction (final)	1.53%	1.36%
LightGBM + time feature	3.04%	2.42%

The lane correction leaves hidden-city loads exactly as the coordinate-based model prices them and improves the rest. An earlier version fed the lane effect to the trees as a feature; they leaned on it instead of the coordinates and hidden-city loads scored 1.85%, which is why the correction is applied after the trees.

Final fit

The final model is refit on all 47,323 clean January–October loads with the same settings, then applied to validation.csv and december_chart_inputs.csv.

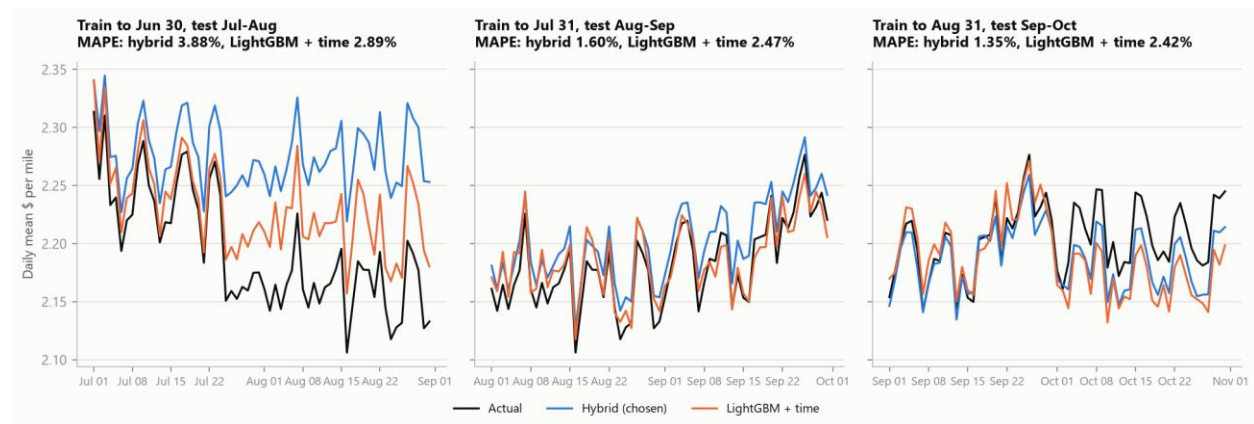


Figure 7. Daily mean rate per mile in each test window: actual against out-of-time predictions.

4. Why this model

- **Log target.** The effects are percentage premiums, so errors are modelled in percent.
- **A parametric stage for anything that must extrapolate.** November–December lie beyond the training dates. Tree models hold the last value they saw for calendar time, so LightGBM with a time feature cannot continue the drift (2.42% on fold 3). A robust (Huber) log-linear model carries the distance curve, equipment and weight premiums, market_index and the drift, and is insensitive to corrupted labels that slip through.
- **Gradient boosting for the rest.** The residual effects are non-linear and interact: the quarter-end ramp differs by equipment, geography is a two-dimensional surface and the quote_signal deviation is U-shaped. LightGBM learns them from coordinates and cyclic calendar features that stay inside their training range in November–December. Days to quarter end are capped at 35, so the trees cannot combine them with the daily market_index to identify and memorise individual training dates.
- **A shrunk lane correction last.** Lanes keep a small specific effect (about 0.6%). Each lane's mean out-of-fold residual is shrunk toward zero (5 pseudo-loads, close to the noise-to-signal variance ratio) and added on top; unseen lanes get zero.
- **Alternatives rejected.** Lane-median \$/mile (no market or time signal), pure linear model (misses the interactions), LightGBM without time (cannot see the drift), LightGBM with time (flatlines after October), and a blend of the hybrid with LightGBM + time (helped one fold, hurt another).

Table 6. Final trend-stage effects.

Effect (trend stage)	Estimate	Reading
Distance elasticity	0.83 at 100 mi, 0.90 at 3,000 mi	Rate grows less than proportionally with miles, so rate per mile falls.
Flatbed vs Dry Van	+8.1%	Constant premium.
Reefer vs Dry Van	+12.9%	Constant premium.
Weight	+3.0% per 10,000 lb	
market_index	+1.45% per +0.1	Same-day value.
Calendar drift	+7.9% per year	Extrapolated into Nov–Dec.

Residual-model importance (share of gain): latitude 43%, longitude 23%, quarter-end window 14%, quote_signal deviation 6%, distance 5%, weight 4%, equipment 3%.

5. December prediction chart

The scorer's fixed inputs vary only the date: Lexington → Fort Wayne, 360 miles, Dry Van, 32,000 lb. Coordinates come from the training city table; market_index and quote_signal for each date are the medians of that day's validation loads.

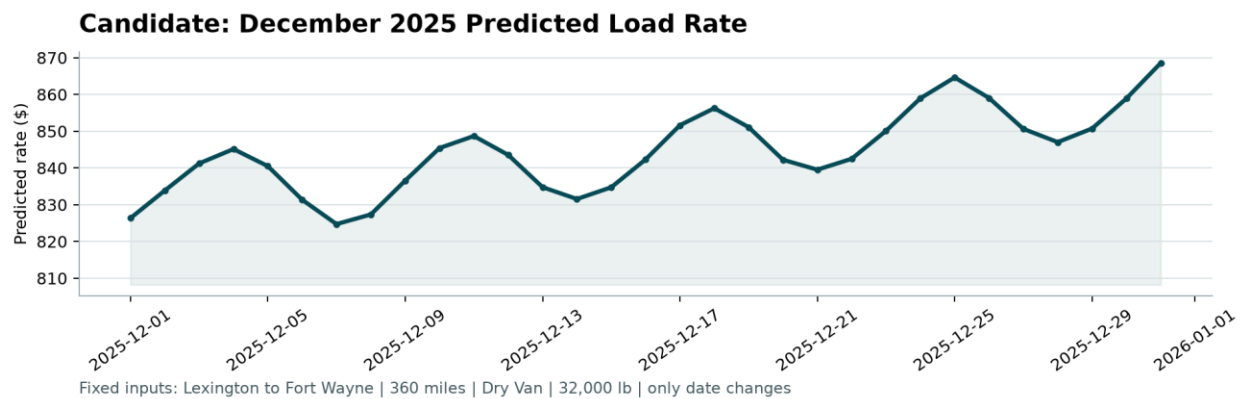


Figure 8. December chart produced by the provided score.py.

- **Range.** \$825–\$869; \$826 on Dec 1 and \$869 on Dec 31.
- **Weekly cycle.** market_index's weekly cycle moves the rate between −1.5% and +1.6%, with peaks on Thursdays (Dec 4, 11, 18, 25).
- **Year-end ramp.** The quarter-close window lifts this load by 1.7% on Dec 31. Across all December validation loads the model adds +2.4% (Dry Van), +7.2% (Flatbed) and +4.7% (Reefer) in the final three days, close to the +2.7%, +8.0% and +5.1% measured in training.
- **Drift.** With both effects removed, the level rises 0.4% over the month.

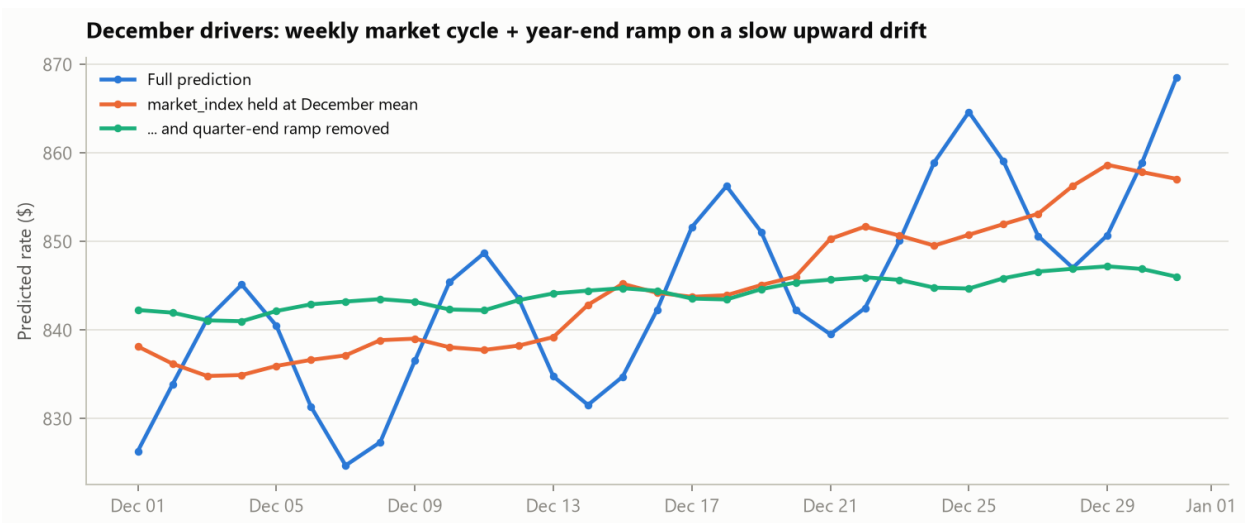


Figure 9. Counterfactuals: holding `market_index` at its December mean removes the weekly cycle; also removing the quarter-end window leaves the drift.

6. Limitations and next steps

- **Holidays.** Thanksgiving and Christmas are absent from training, so no holiday effect is modelled. The Christmas-day peak comes from the Thursday market cycle, not from a learned holiday pattern.
- **Linear drift.** If growth plateaus as it did in July–August, November–December predictions could run about 1% high; the drift estimate moved between 0.063 and 0.084 per year across folds 2–3.
- **Corrupted labels in the scoring set.** If validation contains them at the training rate (1.4%), they will dominate RMSE and add about 2.5 points to MAPE for any model.
- **Laredo** lies south of every training city, so its latitude effect is held at the southernmost training value.
- **Next steps.** Prediction intervals (quantile LightGBM), monitoring of input drift such as `quote_signal`, and monthly retraining so the drift estimate keeps updating.

Appendix A: reproducing the results

```
python -m venv .venv
.venv\Scripts\activate          (macOS/Linux: source .venv/bin/activate)
pip install -r requirements.txt
python eda.py
python train.py
python score.py --predictions validation_predictions.csv --december-predictions
data/december_chart_inputs.csv
python make_report.py
```

Appendix B: key code

The two most important pieces of code, copied from the repository each time this report is built.

Code 1. HybridModel.fit and HybridModel.predict (src/models.py, lines 143–162): trend stage, LightGBM on what is left, then the lane correction.

```
def fit(self, df: pd.DataFrame):
    self.trend = ParametricModel().fit(df)
    resid = np.log(df[TARGET]) - self.trend.predict_log(df)
    X = df[GBM_FEATURES]
    self.gbm = self._gbm(X, resid)
    if self.lane_correction:
        # Out-of-fold stage-2 predictions: in-sample residuals would already
        # be partly fitted by the trees and understate lane effects.
        oof = np.zeros(len(df))
        for fit_idx, pred_idx in KFold(5, shuffle=True, random_state=SEED).split(df):
            fold_model = self._gbm(X.iloc[fit_idx], resid.iloc[fit_idx])
            oof[pred_idx] = fold_model.predict(X.iloc[pred_idx])
        self.lanes = LaneCorrection().fit(df, resid - oof)
    return self

def predict(self, df: pd.DataFrame) -> np.ndarray:
    log_rate = self.trend.predict_log(df) + self.gbm.predict(df[GBM_FEATURES])
    if self.lane_correction:
        log_rate = log_rate + self.lanes.transform(df)
    return np.exp(log_rate)
```

Code 2. time_backtest (src/validation.py, lines 24–49): the date-based, expanding-window split.

```
def time_backtest(df: pd.DataFrame, make_model, outlier_mask: pd.Series) -> list[dict]:
    """Expanding-window backtest.

    Training rows are filtered for label outliers using a mask computed on the
    training window only; test rows are always scored in full ("all") and
    separately with outliers removed ("clean") for a noise-free read.
    """
    results = []
    for train_end, test_start, test_end in BACKTEST_FOLDS:
        train = df[df["date"] <= train_end]
        test = df[(df["date"] >= test_start) & (df["date"] <= test_end)]
        train = train[~flag_label_outliers(train)]
        model = make_model().fit(train)
        pred = model.predict(test)
        clean = ~outlier_mask.loc[test.index]
        results.append(
            {
                "fold": f"train<= {train_end} | test {test_start}..{test_end}",
                "n_train": len(train),
                "n_test": len(test),
                "all": metrics(test[TARGET], pred),
                "clean": metrics(test.loc[clean, TARGET], pred[clean.to_numpy()]),
                "pred": pd.Series(pred, index=test.index),
            }
        )
    return results
```